

$$\begin{matrix} [a & b & c] \\ 1 \times 3 \end{matrix} \begin{matrix} \begin{bmatrix} d \\ e \\ f \end{bmatrix} \\ 3 \times 1 \end{matrix}$$

$$= 1 \times 1 = a \times d + b \times e + c \times f$$

Classic naive GEMM:

→ General Matrices

Multiply.

```
</> C++  
  
for(i)  
  for(j)  
    for(k)  
      C[i][j] += A[i][k] * B[k][j];
```

$i_0 \quad i_1 \quad i_2$

k changing fast

Considering row major this is

sequential travel which is good

but

$K \rightarrow 0 \ 3 \ 1 \ 5 \ 2 \ 5$

are 5 units across memory

So bad locality.

</> C++

```
for(i)
  for(k)
    for(j)
      C[i][j] += A[i][k] * B[k][j];
```

compared to:

</> C++

```
for(i)
  for(j)
    for(k)
```

→ k changes fast
so walk across

row

Good locality

Even Though Same Complexity

Better memory usage.

Cache is small so we do in chunks.

because matrix
is huge.

Tiling is basically:

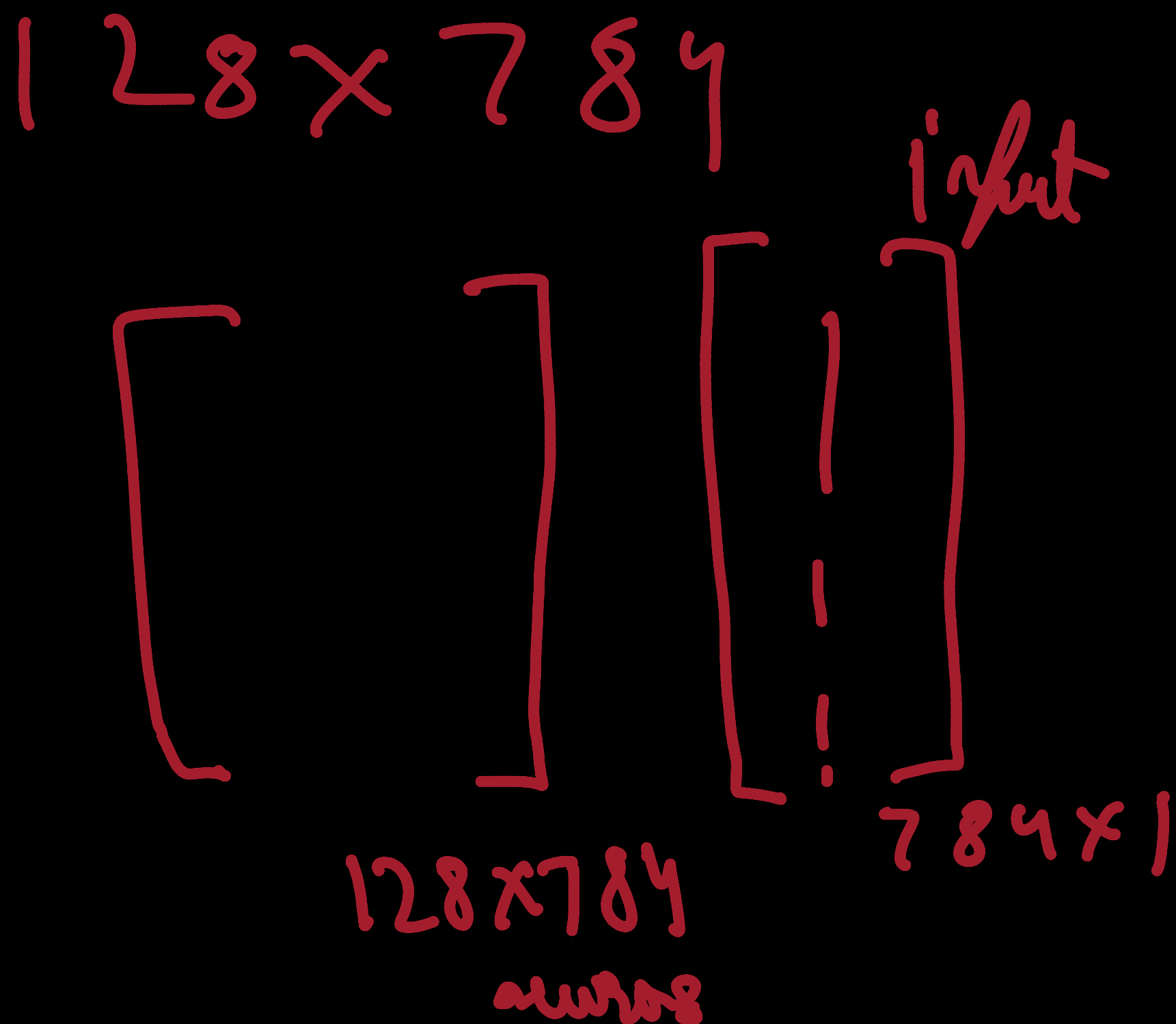
“organize computation so reusable data stays in cache/
registers longer.

• Suppose:

784 input values

128 neurons

What would be the shape of the weight matrix?



SIMP $\Rightarrow$ Single instructions

Multiple data.

Why do Tensors even exist?

Thing

Meaning

data pointer

where memory lives

shape

dimensions

strides

how indexing maps to memory

dtype

float/int/etc

2x3 matrix Shape is 2x3 [2][3]

Stride is

how much memory changes with one
step in any direction.

[2][3]

$[2 \times 3]$

$\rightarrow 1\ 2\ 3\ 4\ 5\ 6$ in memory

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ representation.

if col + 1 $[0] \rightarrow +1$

$[1]$

Changes in memory Just one step ahead

Now row movement

$0\ 1 \rightarrow [1]\ [0]$

move ~~to~~ across elements.

So number of elements

Ex 2×3

$\rightarrow$ So Col

stride $[3 \ 1]$

become row
strides.

$$i \times \text{Col} + J$$

is just Stride

Arithmetic.

- Another important realization

A transpose operation can be:

Cheap logically

(change strides only)

BUT

Expensive computationally later

(bad traversal/locality)